

Mise en œuvre

SmartHome Control

Introduction

Ce document présente l'organisation et la mise en œuvre du projet après validation du cycle en V.

Notre entreprise a été choisie par une grande figure du domaine de la domotique afin de concevoir et développer un système d'automatisation pour maisons connectées. Le système doit permettre aux utilisateurs de gérer et contrôler leurs équipement domestique et d'automatiser certaines tâches à partir d'une interface simple et accessible.

Le projet présente des enjeux importants :

- Sécurité des données et équipements
- Fiabilité des commandes
- Disponibilité du système
- Simplicité d'utilisation
- Évolutivité et maintenance
- Traçabilité des opérations
- Respect des exigences en matière de données personnelles

Après analyse du contexte, le Cycle en V est retenu comme méthodologie de développement.

Ce document va montrer :

- L'analyse du besoin
- La formalisation des exigences
- Transformer ces exigences en spécifications fonctionnelles
- Concevoir une architecture
- Détailler les composants techniques
- Préparer la validation à chaque niveau
- Assurer la traçabilité entre les besoins, la conception et les tests

Le projet sera organiser selon les quatre niveaux de conception du Cycle en V.

Exigences et besoins client :

Cahier des charges

Contexte et Objectif :

Le marché de la maison connecté regroupe de nombreux équipements provenant de constructeurs différents. Cette diversité cause plusieurs problèmes :
Protocoles différents, interfaces différentes, niveaux de sécurité variables, difficultés de centralisation, multiplication des applications, configuration complexe pour les utilisateurs, et bien d'autres encore.

Le projet consiste à concevoir une solution avec une interface centralisée permettant aux occupants d'une maison de :

- Superviser leurs équipements connectés
- Contrôler leurs équipements
- Automatiser certaines actions
- Consulter l'état de leur domicile
- Configurer des scénarios
- Recevoir des alertes

Le système doit être :

- Simple à utiliser
- Fiable
- Sécurisé
- Évolutif
- Accessible depuis une interface web

Le système doit permettre à un utilisateur de :

- Créer un compte
- Se connecter
- Créer un foyer
- Inviter d'autres utilisateurs
- Définir leurs permissions
- Ajouter des équipements
- Consulter leur état
- Contrôler les équipements
- Créer des automatisations
- Programmer des actions
- Recevoir des notifications
- Consulter l'historique

Public cible :

Le système cible principalement :

- Les particuliers possédant plusieurs équipements connectés
- Les utilisateurs souhaitant automatiser leur logement
- Les utilisateurs souhaitant centraliser leurs équipements

Persona 1 - Thomas, utilisateur technophile

Âge : 32 ans

Profil : développeur passionné de technologie

Possède les équipements suivants :

- Éclairage connecté
- Thermostat
- Volets
- Capteurs

Besoins : créer des automatisations complexes

Problème : les différentes applications des ses équipements ne communiquent pas entre elles

Objectif : Centraliser et automatiser son installation

Persona 2 - Sophie, utilisatrice familiale

Âge : 43 ans

Profil : mère de famille, utilisation avec smartphone

Possède les équipements suivants :

- Éclairage connecté
- Chauffage
- Thermostat

Besoins : veut une interface simple pour contrôler ses équipements

Problème : les systèmes sont parfois trop complexes

Objectif : Pouvoir utiliser la domotique sans connaissances particulières

Persona 3 - Marc, utilisateur orienté sécurité

Âge : 43 ans

Profil : propriétaire d'une maison équipée de capteurs

Besoins : veut surveiller les ouvertures, les mouvements, certains équipements

Problème : il veut éviter qu'un utilisateur non autorisé puisse modifier son installation

Objectif : Disposer d'un système fiable avec une gestion précise des permissions

MVP :

Le MVP doit rester suffisamment limité pour permettre une validation réaliste du Cycle en V.

Utilisateurs

- Création de compte
- Connexion
- Déconnexion

Foyers

- Création d'un foyer
- Gestion des membres

Équipement :

- Ajout
- Suppression
- Consultation de l'état
- Commande

Automatisation :

- Création d'une règle simple
- Activation/désactivation
- Déclenchement sur un évènement

Notification :

- Notification d'évènement important

Historique :

- Consultation des évènements

Évolutions possibles :

Les évolutions suivantes sont volontairement exclues du MVP pour rendre la mise en œuvre du Cycle en V plus faisable :

Commande vocale, IA, prédiction de consommation, reconnaissance de présence avancée, analyse énergétique avancée

Cette séparation permet de mieux maîtriser le périmètre initial.

User Stories :

ID	Rôle	Action/besoin
US - 01	En tant que visiteur	Je veux créer un compte afin d'accéder à la plateforme
US - 02	En tant que visiteur	Je veux me connecter à mon profil afin d'utiliser la plateforme
US - 03	En tant que utilisateur	Je veux pouvoir me déconnecter afin de clôturer ma session
US - 04	En tant que utilisateur	Je veux pouvoir modifier mon profil afin de mettre à jour mes informations personnelles et de connexion
US - 05	En tant que propriétaire	Je veux créer un foyer afin de regrouper mes équipements
US - 06	En tant que propriétaire	Je veux ajouter des membres dans mon foyer
US - 07	En tant que propriétaire	Je veux retirer des membres dans mon foyer
US - 08	En tant que propriétaire	Je veux ajouter un équipement afin de pouvoir le contrôler
US - 09	En tant que propriétaire	Je veux supprimer un équipement afin de ne plus l'utiliser
US - 10	En tant que utilisateur autorisé	Je veux consulter l'état d'un équipement afin de connaître son état actuel
US - 11	En tant que utilisateur autorisé	Je veux commander un équipement afin de modifier son état
US - 12	En tant que propriétaire	Je veux créer une règle automatique afin que certaines actions soient exécutées sans intervention manuelle
US - 13	En tant que propriétaire	Je veux activer ou désactiver les actions automatiques
US - 14	En tant que propriétaire	Je veux gérer les permissions des membres afin de contrôler leurs accès
US - 15	En tant que utilisateur	Je veux recevoir une notification lorsqu'un évènement important survient

Exigences fonctionnelles :

ID	Fonction	Exigence	Priorité
EF - 1	Authentification	Le système doit permettre l'authentification sécurisé des utilisateurs	Critique
EF - 2	Permissions	Le système doit gérer les droits d'accès aux équipements	Critique
EF - 3	Équipements	Le système doit permettre d'ajouter et supprimer des équipements	Critique
EF - 4	État	L'utilisateur doit pouvoir consulter l'état des équipements	Critique
EF - 5	Commande	Un utilisateur autorisé doit pouvoir commander un équipement	Critique
EF - 6	Automatisation	Un utilisateur doit pouvoir créer des règles automatisées	Critique
EF - 7	Foyers	L'utilisateur doit pouvoir créer et gérer un foyer	Haute
EF - 8	Membres	Le propriétaire doit pouvoir inviter des membres	Haute
EF - 9	Programmation	L'utilisateur doit pouvoir programmer des actions	Haute
EF - 10	Notifications	Le système doit envoyer des notifications	Haute
EF - 11	Historique	Le système doit conserver l'historique des événements	Haute
EF - 12	Administration	Un administrateur doit disposer d'outils de supervision et de diagnostic	Moyenne

Exigences non fonctionnelles :

ID	Domaine	Exigence
ENF - 1	Sécurité	Toutes les communications doivent être chiffrées
ENF - 2	Authentification	Les mots de passe doivent être stocker sous forme hachée et salée
ENF - 3	Autorisation	Les permissions doivent être contrôlées côté serveur
ENF - 4	Disponibilité	Objectif de disponibilité de 99,9%, hors maintenance planifiée
ENF - 5	Performance	95% des requêtes API courantes doivent répondre en moins de 500ms, hors latence IoT
ENF - 6	Scalabilité	L'architecture doit permettre l'ajout de nouveaux foyers et équipements
ENF - 7	Résilience	Une panne temporaire d'un composant ne doit pas entraîner de perte de données critiques
ENF - 8	Traçabilité	Les opérations sensibles doivent être journalisées
ENF - 9	RGPD	Les données personnelles doivent être minimisées et protégées
ENF - 10	Maintenance	Le code doit être modulaire et couvert par des tests automatisés

Règles de gestion :

ID	Règle
RG - 01	Un utilisateur peut appartenir à plusieurs foyers
RG - 02	Un foyer peu avoir plusieurs équipement
RG - 03	Un équipement appartient à un seul foyer
RG - 04	Un utilisateur ne peut commander qu'un équipement auquel il possède les droits nécessaires
RG - 05	Seul le propriétaire peut gérer les membres du foyer
RG - 06	Une automatisation désactivée ne doit jamais être exécutée
RG - 07	Une action importante doit être enregistrée dans l'historique
RG - 08	Une commande ne doit être considérée comme exécutée que lorsque l'état de l'équipement a été confirmé

Plan de recette :

Le plan de recette définit comment le client vérifiera que le produit répond à ses besoins.

Critères d'acceptation :

ID	Etant donné	Quand	Action
CA - 01	Un utilisateur enregistré	Il saisit ses identifiants	Il accède à son espace
CA - 02	Un utilisateur autorisé	Il commande un équipement	La commande est transmise et l'état est actualisé
CA - 03	Un utilisateur sans permission	Il tente de commander un équipement	La commande est refusée
CA - 04	Une règle active	Son déclencheur survient	L'action configurée est exécutée
CA - 05	Un utilisateur ou le système exécute une action	Une action importante est exécutée	Elle apparaît dans l'historique

Registre des risques :

ID	Risque	Probabilité	Impact	Intensité
R - 01	Compromission d'un compte	XL	Très élevée	Critique
R - 02	Équipement incompatible	M	Élevé	Élevé
R - 03	Perte de connexion IoT	L	Élevé	Élevé
R - 04	Dérive du périmètre	M	Élevé	Élevé
R - 05	Exigences mal comprises	M	Élevé	Élevé
R - 06	Performances insuffisantes	F	Moyen	Faible
R - 07	Fuite de données	XL	Très élevée	Critique

Validation de la phase 1 :

La phase 1 est considérée comme validée quand :

- cahier des charges validé
- Personas validées
- MVP validé
- User stories validées
- Critères de recette sont définis
- Risques initiaux identifiés

Spécifications fonctionnelles :

Dans cette partie nous allons transformer les besoins exprimés lors de la phase 1 en comportement fonctionnels précis et compréhensible par l'ensemble des parties prenantes;
La problématique est : Comment le système doit-il se comporter du point de vue utilisateur ?

Cahier des charges fonctionnel :

Le cahier des charges fonctionnel décrit de manière détaillée les fonctions que le système doit fournir et le comportement attendu de ces fonctions.

Contrairement au premier cahier des charges de la phase 1, Il ne présente plus le besoin métier général mais les contraintes fonctionnelles et règles de gestion détaillées.

Il précise les acteurs, les fonctionnalités, les règles applicables, les cas d'erreur, les droits nécessaires, les dépendances entre fonctionnalités.

Il est indépendant des choix techniques.

Acteurs fonctionnels :

Le système comporte quatre types s'acteurs:

Acteur	Description	Actions principales
Visiteur	Personne ne possédant pas encore de compte	Créer un compte, se connecter
Utilisateur	Utilisateur possédant un compte et membre d'un foyer	Consulter les équipements, utiliser les fonctionnalités autorisées
Propriétaire	Utilisateur responsable du foyer	Gérer le foyer, les membres, équipements et autorisations
Administrateur	Utilisateur chargé de la supervision de la plateforme	Superviser et diagnostiquer le système

Remarque :

Un foyer peut avoir plusieurs propriétaires.

Un même utilisateur peut être propriétaire et membre d'un autre foyer.

Les permissions sont donc déterminées au niveau de l'appartenance à un foyer.

Fonctionnalités :

F - 01 Création d'un compte :

Le visiteur renseigner les informations nécessaires à la création de son compte.

Le système vérifie :

- Que les champs obligatoires sont renseignés
- Que l'adresse email respect le format attendu
- Que l'adresse email ne soit pas déjà utilisée
- Que les informations respectent les règles définies
- Que le mot de passe soit sécurisé (Regex solides)
- Que les champs mot de passe et confirmation du mot de passe correspondent.

Le nouvel utilisateur peut s'authentifier sur la plateforme après cela.

Cas d'erreur :

- Adresse email déjà utilisée
- Informations invalide
- Mauvais format de l'adresse email
- Tous les champs obligatoires ne sont pas remplis
- Les mots de passe ne correspondent pas
- Le format du mot de passe ne correspond pas au format attendu (Regex)

F - 02 Connexion

Permettre à un utilisateur enregistré d'accéder à son espace personnel.

Le système vérifie les informations fournies. Si elles sont valides, une session utilisateur est ouverte.

L'utilisateur est ensuite redirigé vers son espace.

Cas d'erreur :

- identifiants incorrects
- Compte inexistant
- Compte inaccessible

F - 03 Déconnexion

Permettre à l'utilisateur de clôturer sa session. L'utilisateur sélectionne l'action de déconnexion.

Le système invalide la session en cours.

L'utilisateur revient à l'écran de connexion.

F - 04 Modification du profil

Permettre à un utilisateur de modifier ses informations personnelles autorisées. L'utilisateur accède à son profil et modifie les informations disponibles. Le système vérifie les nouvelles valeurs avant de les enregistrer. Les informations du profil sont mises à jour.

Cas d'erreur :

- Valeur invalide
- Donnée obligatoire supprimée
- Conflit avec une donnée existante

F - 05 Création d'un foyer

Permettre à un utilisateur de créer un espace représentant son logement. L'utilisateur renseigne le nom du foyer. Le système crée le foyer et désigne automatiquement l'utilisateur comme propriétaire. Le nouveau foyer apparaît dans l'espace utilisateur.

F - 06 Ajouter un membre

Permettre au propriétaire d'inviter un autre utilisateur à rejoindre son foyer. Le propriétaire renseigne l'identifiant ou l'adresse du compte à inviter. Il définit les permissions associées au membre. Le système crée une invitation ou une association de membre. Le membre dispose des permissions définies après acceptation de l'invitation.

F - 07 Retirer un membre

Permettre au propriétaire de retirer un utilisateur du foyer. Le propriétaire sélectionne un membre puis demande sa suppression. Le système vérifie que l'utilisateur effectuant l'action est bien propriétaire. L'appartenance au foyer est supprimée. Le membre ne peut plus accéder aux ressources du foyer.

Cas d'erreur :

- Utilisateur non propriétaire
- Membre inexistant
- Tentative de suppression du propriétaire

F - 08 Ajouter un équipement

Permettre au propriétaire d'ajouter un équipement à son foyer. Un équipement possède au minimum :

- un nom
- Un type
- Un foyer
- Un état
- Une disponibilité

Le propriétaire sélectionne l'ajout d'un équipement et renseigne les informations demandées. Le système vérifie les informations puis associe l'équipement au foyer. L'équipement apparaît dans la liste des équipements du foyer.

F - 09 Supprimer un élément

Permettre au propriétaire de retirer un équipement du foyer. Le propriétaire sélectionne l'équipement puis confirme sa suppression. Le système vérifie ses droits avant d'effectuer l'opération. L'équipement n'est plus disponible dans le foyer.

F - 10 Consulter l'état d'un équipement

Permettre à un utilisateur autorisé de connaître l'état actuel d'un équipement.

Selon le type d'équipement :

- état marche/arrêt
- température
- niveau
- disponibilité
- dernière mise à jour

L'utilisateur obtient l'état connu du système.

L'état doit être clairement distingué entre :

- équipement disponible
- équipement indisponible
- état connu
- état en attente de confirmation.

F - 11 Commander un équipement

Permettre à un utilisateur autorisé de modifier l'état d'un équipement.

L'utilisateur sélectionne l'équipement. Il sélectionne une commande. Le système vérifie ses permissions. Le système vérifie la validité de la commande. La commande est transmise. Le système attend la confirmation de l'état. Le nouvel état est affiché.

L'équipement applique la commande et son état confirmé est affiché.

Cas d'erreur :

- utilisateur non autorisé
- équipement inexistant
- équipement indisponible
- commande invalide
- absence de confirmation

F - 12 Créer une automatisation

Permettre au propriétaire de créer une règle permettant d'exécuter automatiquement une action.

Une automatisation comporte

- un nom
- un déclencheur
- éventuellement une condition
- une ou plusieurs actions
- un état actif/inactif.

Exemple :

Déclencheur : La température est en dessous de 19°C

Condition : Entre 18h00 et 23h00

Action : Activer le chauffage

La règle est enregistrée et peut être activée.

F - 13 Activation et désactivation

Permettre au propriétaire de contrôler l'exécution d'une automatisation.

Le propriétaire peut modifier l'état :

Actif ou inactif

Une automatisation inactive ne doit générer aucune action.

F - 14 Gérer les permissions

Permettre au propriétaire de contrôler les actions accessibles aux membres.

Rôle	Consultation	Commande	Autorisation	Membres
Propriétaire	Oui	Oui	Oui	Oui
Membre autorisé	Oui	Selon permission	Non	Non
Membre lecture seule	Oui	Non	Non	Non

Les droits sont appliqués côté serveur.

F - 15 Notification

Informar l'utilisateur lorsqu'un événement important survient.

Exemples :

- équipement indisponible
- automatisation exécutée
- problème d'exécution
- événement de sécurité.

Une notification est générée et rendue accessible à l'utilisateur concerné.

F - 16 Administration

L'administrateur peut accéder à des informations permettant de superviser la plateforme.

Les fonctionnalités comprennent notamment :

- consultation de l'état général du système
- consultation des erreurs
- consultation des événements techniques
- identification des équipements ou services indisponibles.

Cette fonctionnalité est de priorité moyenne et n'est pas nécessairement exposée aux utilisateurs classiques.

F - 17 Historique

Bien que l'historique ne possède pas de User Story distincte dans la phase 1, il constitue une fonctionnalité du MVP.

Le système conserve les événements importants.

Date	Utilisateur	Équipement	Action	Résultat
12/06/2027 22:36	Thomas	Lampe salon	On	Confirmé
13/06/2027 : 20:03	Système	Chauffage	On	Confirmé

Équipe projet :

Rôle	Responsabilité
Chef de projet / PO	Coordination, besoins, validation
Architecte	Modélisation, cohérence fonctionnelle
UX/UI Designer	Wireframes et maquettes
Développeurs Back-end	Analyse de faisabilité fonctionnelle
Développeur Front-end	Analyse des interfaces
QA/Testeurs	Stratégie et cas de tests

Planification des tâches :

ID	Tâche	Livrable	Responsable	Dépendance	Durée
T - 01	Analyse des exigences phare	Base de spécification CDC	Chef de projet	Phase 1	1j
T - 02	Définition des acteurs	Acteurs fonctionnels	Chef de projet + Architecte	T - 01	0.5j
T - 03	Décomposition des fonctionnalités	CDC fonctionnel	Chef de projet + Architecte	T - 01	2j
T - 04	Définition des parcours utilisateur	Parcours	Chef de projet + Architecte	T - 02	1j
T - 05	Wireframes	Wireframe	UX/UI	T - 04	1j
T - 06	Validation Wireframes	Compte rendu wireframe	Chef de projet + Client	T - 05	0.5j
T - 07	Maquettes	Maquettes	UX/UI	T - 05	3j
T - 08	Validation Maquettes	Compte rendu maquettes	Chef de projet + Client	T - 07	0.5j
T - 09	Cas d'utilisation	UML Use case	Architecte	T - 03	1.5j
T - 10	Diagramme de séquence	UML Séquence	Architecte	T - 08	2j
T - 11	Modélisation des données	MCD	Architecte	T - 09	2j
T - 12	Stratégie de tests fonctionnels	Plan de tests	QA	T - 09 + T - 10	2j
T - 13	Plan de validation	Plan de validation	QA + Chef de projet	T - 12	1j
T - 14	Revue de cohérence	Revue phase 2	Toute l'équipe	Tous	1j
T - 15	Validation finale	Jalon V2	Client + Chef de projet	T 14	0.5j

Wireframes :

Maquettes :

Diagramme de cas d'utilisation :

Diagramme de séquence :

MCD :

Stratégie de tests fonctionnels / E2E :

Validation de la phase 2 :

La phase 2 est considérée comme validée lorsque

- le cahier des charges fonctionnel est complet
- toutes les User Stories du MVP sont couvertes
- les fonctionnalités sont suffisamment détaillées
- les wireframes sont validés
- les maquettes sont validées cohérentes avec les wireframes
- les diagrammes de cas d'utilisation sont validés
- les scénarios principaux sont représentés par des diagrammes de séquence
- le MCD est cohérent avec les fonctionnalités
- la stratégie de tests fonctionnels end-to-end est définie
- les règles de gestion sont couvertes par des tests
- aucune incohérence majeure n'est identifiée.

Conception Générale :

La conception générale a pour objectif de définir comment le système sera construit afin de répondre aux exigences fonctionnelles définies lors de la phase précédente.

Cette phase permet notamment de :

- comparer les solutions techniques existantes
- sélectionner une solution adaptée au projet
- définir l'architecture logicielle globale
- identifier les différents composants du système
- définir les responsabilités de chaque composant
- définir les échanges entre les composants
- sélectionner les technologies nécessaires
- prendre en compte les contraintes de sécurité, de performance, de maintenance et d'évolution.

Le choix de l'architecture ne doit donc pas être effectué uniquement en fonction des technologies disponibles, mais en fonction des besoins fonctionnels, des contraintes du projet et de sa capacité d'évolution.

Infrastructure :

Diagramme de déploiement

Benchmark technologique :

JavaScript / TypeScript — retenu

Le choix de Node.js avec TypeScript est le plus cohérent pour ce projet. Il permet de développer l'API REST avec un langage fortement typé grâce à TypeScript, tout en conservant un écosystème web très riche. Il permet également d'utiliser JavaScript/TypeScript côté frontend et backend, ce qui réduit le nombre de technologies à maîtriser.

PHP — très bonne alternative

PHP reste particulièrement pertinent pour le développement web, notamment avec Symfony. Il aurait constitué un excellent choix pour une architecture backend structurée et durable. Il est cependant moins intéressant ici que Node.js/TypeScript si l'objectif est d'avoir une continuité technologique avec le frontend.

Python — très bon mais moins adapté

Python est excellent pour les API, l'automatisation, le traitement de données et l'IA. Pour une application web classique, il n'apporte cependant pas d'avantage déterminant par rapport à Node.js ou PHP. Il est donc écarté pour limiter la diversité technologique.

Java — robuste mais disproportionné

Java offre d'excellentes performances, une forte robustesse et des outils adaptés aux applications d'entreprise. Cependant, son écosystème et sa structure sont plus lourds pour un projet de cette taille. Le coût en complexité ne se justifie pas ici.

Go — performant et léger

Go est particulièrement intéressant pour les API, les services réseau et les applications nécessitant de bonnes performances avec une faible consommation de ressources. Il est néanmoins moins pertinent ici que TypeScript, notamment pour l'écosystème frontend/backend commun.

Choix final

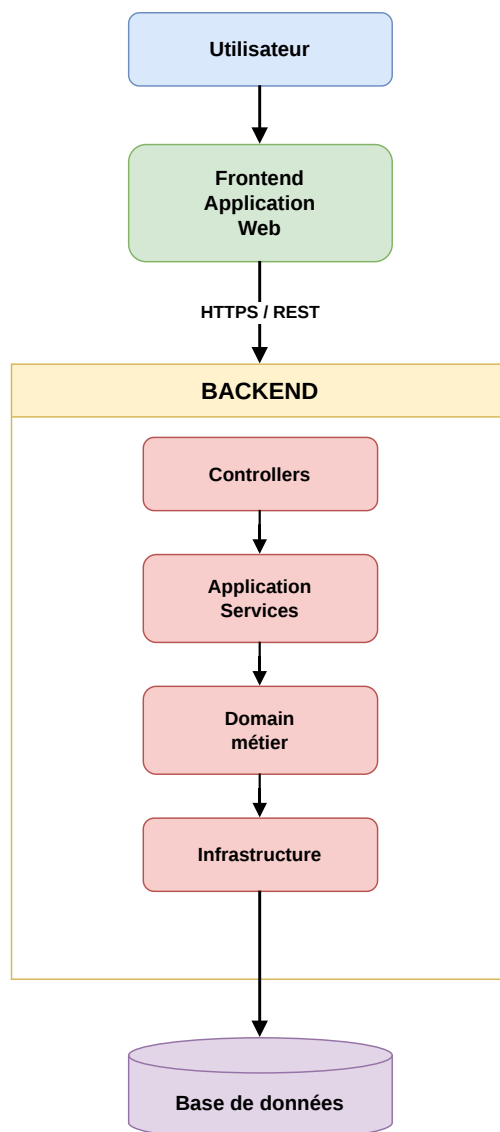
Node.js + TypeScript est retenu pour le backend.

Ce choix permet d'obtenir un bon équilibre entre performance, simplicité, typage, maintenabilité, écosystème et évolutivité, tout en restant cohérent avec le choix de React pour le frontend.

Technologie	Performance	Explications	Décision
JavaScript / TypeScript	Très bonne	Choix cohérent, API REST, langage typé, écosystème riche, JS côté frontend et backend Dynamique et asynchrone Facile à maîtriser	Retenu
PHP / Symfony	Très bonne	Architecture structurée et durable. Technologies frontend et backend différentes	Alternative
Python	Très bonne	Excellent pour API, automatisation et traitement de données N'apporte pas davantage que JS et PHP	Écarté
Java	Très bonne	Excellentes performances, robuste, adapté aux applications d'entreprises Mais eco système et structure lourde Coût en complexité	Rejeté
Go	Excellente	Très bon pour API services réseau et applications Bonne performances pour faibles ressources Mais écosystème frontend et backend non commun	Alternative

MLD :

Architecture logicielle :



Les couches :

Domaine : contient les règles métier essentielles

Application : orchestre les cas d'utilisation, décrit ce que l'application doit faire

Infrastructure : fournit les implémentations techniques nécessaires au fonctionnement de l'application

Contraintes techniques :

Stratégie de tests d'intégration :

Validation de la phase 3 :

La phase 3 est considérée comme validée lorsque

- L'infrastructure est décidée et validée
- Un benchmark des stacks technologiques a été réfléchi et validé
- Le MLD est validé
- L'architecture logicielle est validée

Conception détaillée :

Diagramme de composant :

Diagramme d'état :

Diagramme de classes :

MPD :

Contrat API :

Règle métier détaillées :

Stratégie de maintenance :

La stratégie de maintenance définit les méthodes permettant de maintenir le système **fonctionnel, sécurisé, disponible et évolutif après sa mise en production**.

La maintenance est particulièrement importante pour un système domotique car le logiciel doit continuer à communiquer avec des équipements connectés pouvant évoluer indépendamment de la plateforme.

Elle doit notamment permettre de :

- corriger les anomalies
- appliquer les mises à jour de sécurité
- maintenir la compatibilité avec les équipements
- prévenir les régressions
- améliorer les performances
- faire évoluer les fonctionnalités
- maintenir l'infrastructure
- assurer la disponibilité du service.

La stratégie repose sur trois types de maintenance :

- maintenance corrective
- maintenance préventive
- maintenance évolutive

Type	Actions principales
Corrective	Correction des bugs et incidents, tests de régression
Préventive	Mises à jour de sécurité, dépendances, sauvegardes et surveillance
Évolutive	Ajout de fonctionnalité et prise en charge de nouveaux équipements

Plan de tests unitaires :

ID	Composant	Scénario	Résultat attendu
TU - 01	Authentification	Identifiants valides	Authentification acceptée
TU - 02	Authentification	Mauvais mot de passe	Authentification refusée
TU - 03	Authentification	Utilisateur inexistant	Authentification refusée
TU - 04	Validation	Email invalide	Validation refusée
TU - 05	Validation	Mot de passe invalide	Validation refusée
TU - 06	Foyer	Création d'un foyer	Foyer crée
TU - 07	Membres	Ajout par propriétaire	Ajout accepté
TU - 08	Membres	Ajout par membre	Action refusée
TU - 09	Permissions	Permission accordée	Action autorisée
TU - 10	Permissions	Permission absente	Action refusée
TU - 11	Équipements	Données valides	Équipement crée
TU - 12	Équipements	Données invalides	Création refusé
TU - 13	Commande	Commande valide	Commande acceptée
TU - 14	Commande	Utilisateur non autorisé	Commande refusée
TU - 15	Commande	Équipement indisponible	Commande refusée ou mise en attente
TU - 16	Automatisation	Déclencheur valide	Règle déclenchée
TU - 17	Automatisation	Condition vraie	Action exécutée
TU - 18	Automatisation	Condition fausse	Action non exécutée
TU - 19	Automatisation	Règle inactive	Aucune action
TU - 20	Notification	Évènement important	Notification générée
TU - 21	Historique	Action importante	Évènement crée
TU - 22	Historique	Action non importante	Aucun événement inutile crée

Validation de la phase 4 :

La phase 4 est considérée comme validée lorsque

- Les diagrammes de composant ont été validés
- Les diagrammes d'état ont été validés
- Les diagrammes de classe ont été validés
- Le MPD a été validé
- Les tickets sont prêts à être développés selon les DoR
- Le plan de mise à jour et de maintenance a été validés
- La stratégie de tests unitaires a été validée